# Terenac

# OpenCourt

## Mobilna aplikacija za rezervaciju teniskih terena i povezivanje sportske zajednice

## Arhitekturni projekat

**Verzija 1.0**

# Pregled izmena

| Datum | Verzija | Opis | Autor |
|---|---|---|---|
| 10.10.2025 | 1.0 | Inicijalna verzija | Luka, Ksenija, Aleksa |
| | | | |
| | | | |
| | | | |

# Sadržaj

1. Cilj dokumenta 4

2. Opseg dokumenta 4

3. Reference 4

4. Predstavljanje arhitekture 4

5. Ciljevi i ograničenja arhitekture 4

6. Pogled na slučajeve korišćenja 5
6.1. Dijagrami slučajeva korišćenja 5
6.2. Kratak opis slučajeva korišćenja 5
6.2.1. Registracija na sistem 5
6.2.2. Prijava na sistem 5
6.2.3. Pregled klubova 5
6.2.4. Pregled terena kluba 5
6.2.5. Rezervacija termina 5
6.2.6. Iznajmljivanje opreme 6
6.2.7. Pregled profila i statistike 6
6.2.8. Pregled rang liste 6
6.2.9. Kreiranje oglasa 6
6.2.10. Pregled i komentarisanje oglasa 6
6.2.11. Chat sa administratorima 6
6.2.12. Dodavanje kluba (Admin) 6
6.2.13. Dodavanje terena (Admin) 6
6.2.14. Upravljanje rezervacijama (Admin) : Kratak opis: Administrator pregleda i upravlja rezervacijama terena svog kluba 7

7. Pogled na logičku arhitekturu sistema 7
7.1. Pregled arhitekture – organizacija paketa i podsistema u slojeve 7

8. Pogled na procese 8
8.1. Procesi 8

9. Pogled na rasporedjivanje sistema 10

10. Pogled na implementaciju sistema 11

11. Performanse 11

12. Kvalitet 12

# Arhitekturni projekat

## 1. Cilj dokumenta

Cilj ovog dokumenta je detaljan opis arhitekture OpenCourt mobilne aplikacije, sistema namenjenog rezervaciji teniskih terena, upravljanju rezervacijama, kreiranju oglasa i izgradnji sportske zajednice. Dokument će služiti kao osnova za implementaciju, održavanje i dalji razvoj sistema.

## 2. Opseg dokumenta

Dokument se odnosi na OpenCourt Android aplikaciju koja će biti razvijena kao studentski projekat. OpenCourt omogućava korisnicima pregled teniskih klubova, rezervaciju termina, iznajmljivanje opreme, praćenje statistike, kreiranje oglasa i direktnu komunikaciju sa administratorima klubova putem chat-a.

## 3. Reference

Spisak korišćene literature:

1. OpenCourt -- Vizija sistema, V1.0, 2025, [Terenac].

2. OpenCourt -- Specifikacija zahteva, V1.0, 2025, [Terenac].

## 4. Predstavljanje arhitekture

Arhitektura sistema u dokumentu je prikazana kao serija pogleda na sistem: pogled na slučajeve korišćenja, pogled na logičku arhitekturu sistema, pogled na procese, pogled na razmeštaj komponenti sistema i pogled na implementaciju. Ovi pogledi su predstavljeni odgovarajućim UML dijagramima.

## 5. Ciljevi i ograničenja arhitekture

Klučni zahtevi i sistemska ograničenja koja imaju značajan uticaj na izbor arhitekture i projektovanje sistema su:

1. **Platforma:** OpenCourt će biti implementiran kao Android mobilna aplikacija korišćenjem Kotlin programskog jezika
2. **Backend:** Sistem koristi Firebase cloud infrastrukturu:

- Firebase Authentication (autentifikacija korisnika)
- Cloud Firestore (NoSQL baza podataka)
- Firebase Storage (opciono za buduće verzije)

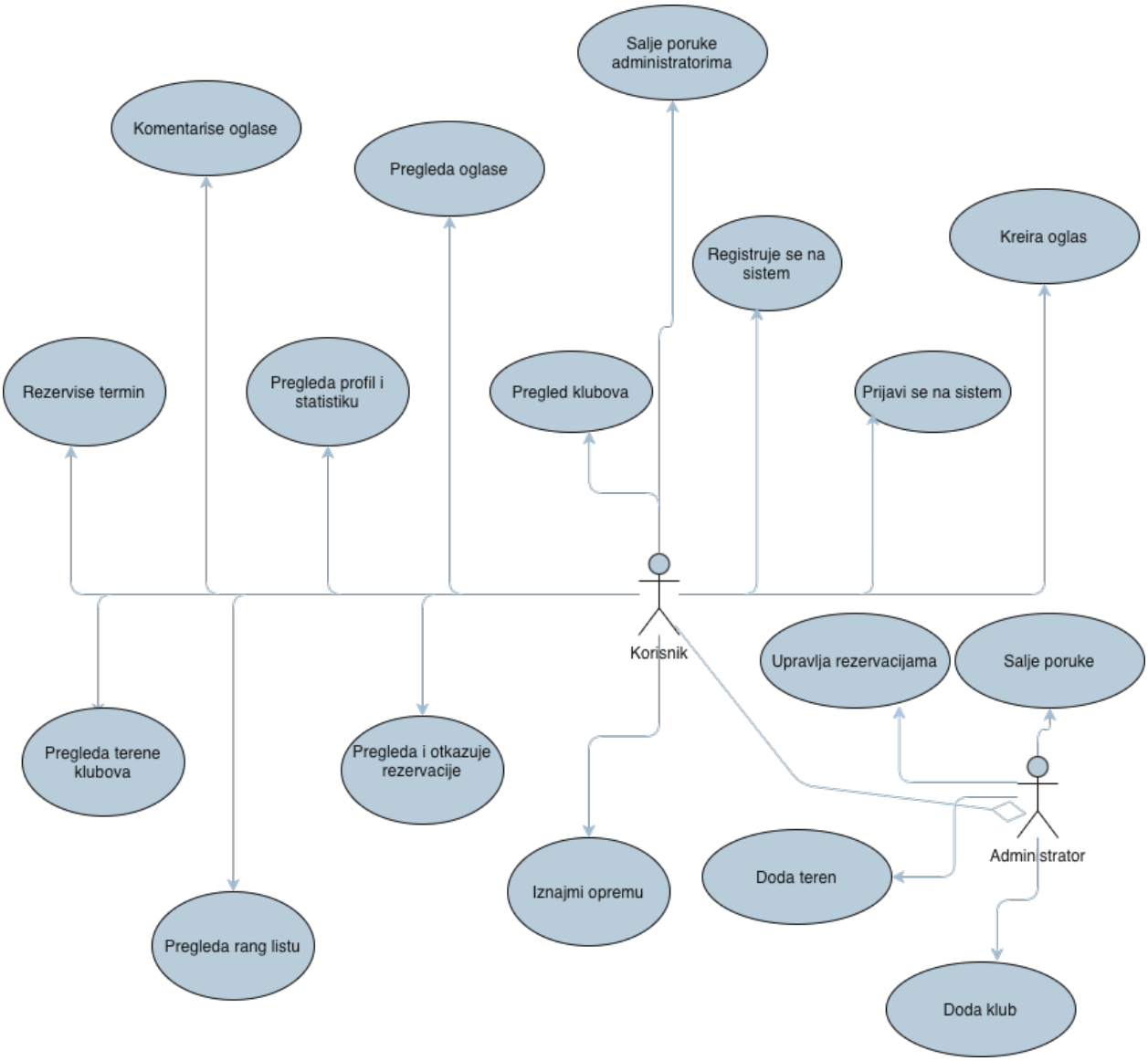
3. **Minimalna verzija:** Android 13.0 (Oreo, API 33) i novije verzije.
4. Podaci (rezervacije, poruke, oglasi) moraju biti sinhronizovani u realnom vremenu putem Firestore listener-a.

5. **Arhitekturni pattern**: MVVM (Model-View-ViewModel) sa Repository pattern-om za pristup podacima.

6.. **GPS i Google Maps:** Integracija Google Maps API-ja za prikaz lokacija klubova i sortiranje po udaljenosti.

# 6. Pogled na slučajeve korišćenja

## 6.1. Dijagrami slučajeva korišćenja




## 6.2. Kratak opis slučajeva korišćenja

### *6.2.1. Registracija na sistem*

Kratak opis: Novi korisnik se registruje unosenjem licnih podataka

### *6.2.2. Prijava na sistem*

Kratak opis: Korisnik se prijavljuje emailom i lozinkom.

### *6.2.3. Pregled klubova*

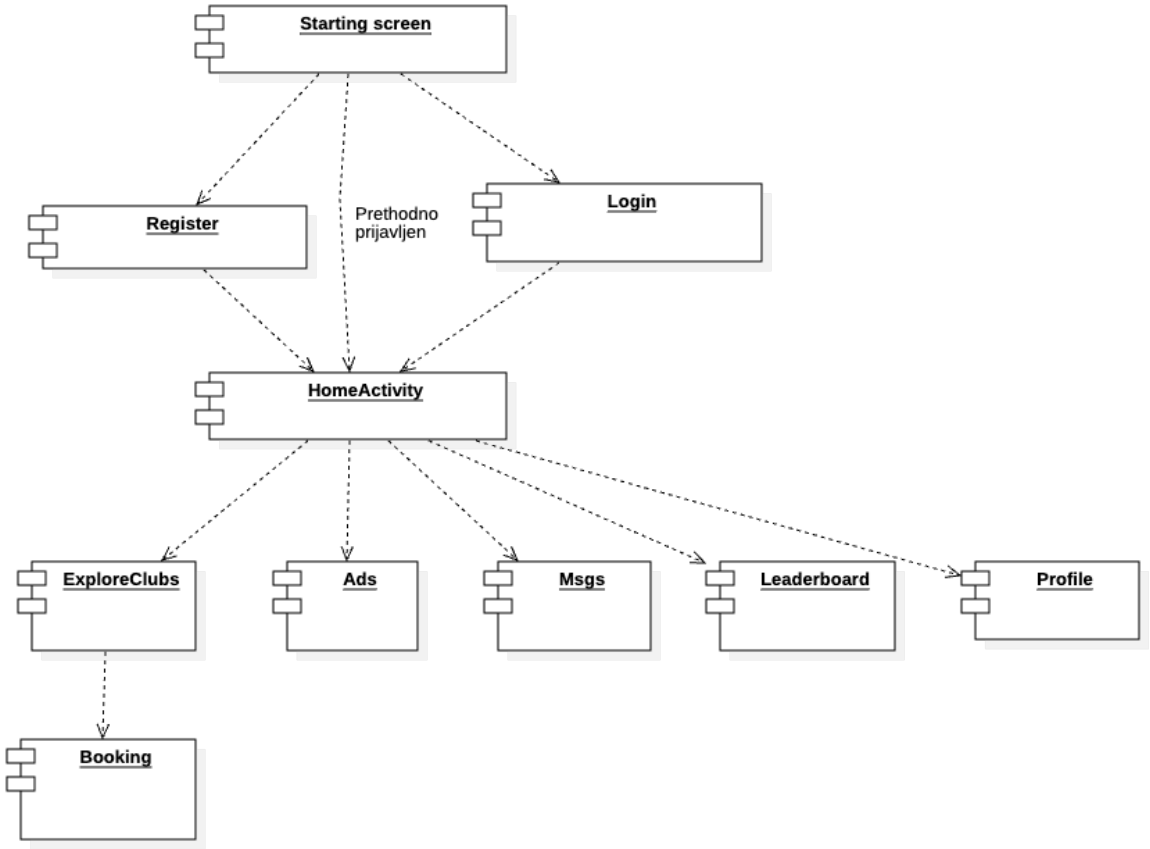
Kratak opis: Korisnik pregleda listu teniskih klubova sa pretragom i sortiranjem.

### *6.2.4. Pregled terena kluba*

Kratak opis: Korisnik pregleda sve terene odabranog kluba sa filtriranjem

### *6.2.5. Rezervacija termina*

Kratak opis: Korisnik rezerviše termin na odabranom terenu




*6.2.6. Iznajmljivanje opreme*

Kratak opis: Korisnik iznajmljuje sportsku opremu (reketi, loptice)

*6.2.7. Pregled profila i statistike*

Kratak opis: Korisnik pregleda lične podatke, bodove i poziciju na rang listi

*6.2.8. Pregled rang liste*

Kratak opis: Korisnik pregleda rang listu svih korisnika sortiranu po bodovima.

*6.2.9. Kreiranje oglasa*

Kratak opis: Korisnik kreira oglas za pronalaženje partnera ili organizaciju meč

*6.2.10. Pregled i komentarisanje oglasa*

Kratak opis:Korisnik pregleda oglase i dodaje komentare

*6.2.11. Chat sa administratorima*

Kratak opis: Korisnik komunicira sa administratorima klubova putem chat-a

*6.2.12. Dodavanje kluba (Admin)*

Kratak opis: Administrator dodaje novi teniski klub u sistem

*6.2.13. Dodavanje terena (Admin)*

Kratak opis: Administrator dodaje novi teren u postojeci klub

*6.2.14. Upravljanje rezervacijama (Admin) : Kratak opis: Administrator pregleda i upravlja rezervacijama terena svog kluba*

# 7. Pogled na logičku arhitekturu sistema

## 7.1. Pregled arhitekture – organizacija paketa i podsistema u slojeve

OpenCourt sistem je organizovan prema **MVVM (Model-View-ViewModel)** arhitekturnom patternu sa sledećim slojevima:

### 7.2. Prezentacioni sloj (View)

Sadrži sve UI komponente aplikacije:

- **Activities:**
  - `MainActivity` — početni ekran i navigacija
  - `HomeActivity` — prikaz liste klubova
  - `ExploreActivity` — prikaz terena odabranog kluba
  - `BookingActivity` — rezervacija termina
  - `ProfileActivity` — profil korisnika i statistika
  - `LeaderBoardActivity` — rang lista korisnika
  - `AdvertisementActivity` — pregled oglasa
  - `CreateAdActivity` — kreiranje novog oglasa
  - `AdPreviewActivity` — detaljan prikaz oglasa
  - `MessagesActivity` — chat sa administratorima
  - `MessagesHistory` — lista svih chat konverzacija
  - `AddClubActivity` — dodavanje kluba (Admin)
  - `AddTenisCourtActivity` — dodavanje terena (Admin)

- **Fragments:**
  - `EquipmentDialogFragment` — dijalog za iznajmljivanje opreme
- **Adapters:**
  - `ClubsAdapter` — RecyclerView adapter za listu klubova
  - `CourtAdapter` — GridView adapter za terene
  - `ReservationsAdapter` — adapter za listu rezervacija
  - `LeaderboardAdapter` — adapter za rang listu
  - `AdAdapter` — adapter za listu oglasa
  - `CommentsAdapter` — adapter za komentare
  - `MessagesAdapter` — adapter za chat poruke

- **Domen sloj (Model)**

Sadrži data klase koje predstavljaju poslovne entitete:

- `User` — korisnik sistema
- `Club` — teniski klub
- `Court` — tenis teren
- `Reservation` — rezervacija termina
- `Advertisement` — oglas
- `Comment` — komentar na oglasu
- `Message` — chat poruka
- `Equipment` — iznajmljena oprema

**Sloj za pristup podacima (Repository)**

Upravlja pristupom Firebase Firestore bazi i Firebase Authentication servisu:

- **Firebase Authentication** — registracija, prijava, logout
- **Cloud Firestore Collections:**
  - `users` — podaci o korisnicima
  - `administrators` — lista admin korisnika
  - `clubs` — podaci o klubovima
  - `clubs/{clubId}/courts` — tereni u klubu
  - `clubs/{clubId}/courts/{courtId}/reservations` — rezervacije za teren
  - `advertisements` — oglasi
  - `chatThreads` — chat konverzacije
  - `chatThreads/{threadId}/messages` — poruke u konverzaciji

**Utility sloj**

Pomoćne klase i funkcionalnosti:
- `Prefs` — SharedPreferences helper za lokalno čuvanje podataka
- `GridSpacingItemDecoration` — helper za Grid layout spacing
- **Google Maps API** — integracija za prikaz lokacija
- **Google Places API** — autocomplete za unos adresa

# 8. Pogled na procese

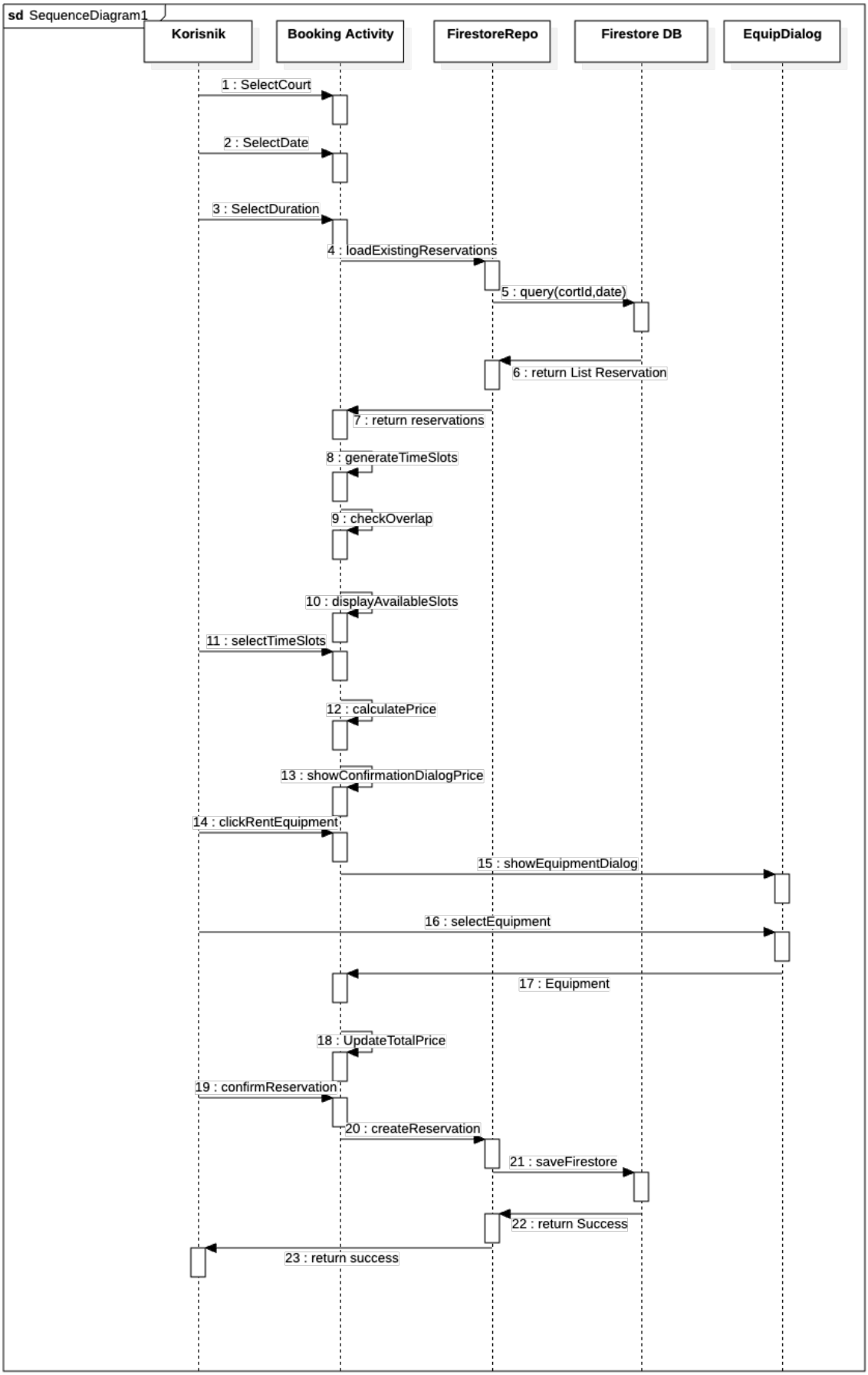
## 8.1. Procesi

OpenCourt aplikacija koristi sledeće glavne procese:

**Proces autentifikacije**

1. Korisnik unosi email i lozinku
2. Aplikacija šalje podatke Firebase Authentication servisu
3. Firebase vraća token i userId
4. Aplikacija učitava korisničke podatke iz Firestore `users` kolekcije
5. Aplikacija proverava da li korisnik postoji u `administrators` kolekciji
6. Sesija se čuva u SharedPreferences

**Proces rezervacije termina**



1. Korisnik odabere klub → teren → datum → trajanje → broj igrača
2. Aplikacija učitava postojeće rezervacije za odabrani datum
3. Sistem generiše listu vremenskih slotova (09:00 - 21:00)
4. Sistem proverava overlap sa postojećim rezervacijama
5. Sistem filtrira prošle slotove ako je datum danas
6. Korisnik odabere dostupan slot
7. Sistem kreira rezervaciju u Firestore bazi
8. Sistem dodeljuje bodove korisniku
9. Rezervacija se kopira u `users/{userId}/reservations`

**Proces real-time chat-a**

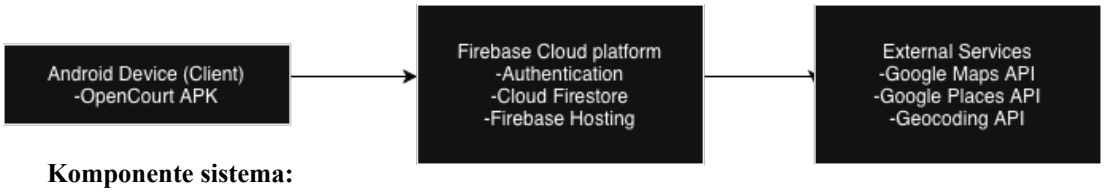
1. Korisnik klikne na "Poruke" tab
2. Aplikacija učitava sve chat threadove iz `chatThreads` kolekcije
3. Korisnik odabere konverzaciju
4. Aplikacija postavlja Firestore `addSnapshotListener` na `messages` sub-kolekciju
5. Nove poruke se automatski prikazuju u realnom vremenu
6. Korisnik unosi tekst i klikne "Pošalji"
7. Poruka se dodaje u Firestore sa timestamp-om
8. Listener detektuje novu poruku i ažurira UI

**Proces sinhronizacije podataka**

- Svi podaci se čuvaju u Cloud Firestore bazi
- Firestore automatski sinhronizuje podatke sa klijentom
- Offline persistence omogućava rad sa keširanih podataka kada nema interneta
- Promene se automatski sinhronizuju kada se internet vrati

## 9. Pogled na rasporedjivanje sistema

OpenCourt sistem koristi cloud-native arhitekturu sa Firebase platformom.




**Komponente sistema:**

1. **Android uređaj (Client)**
   - OpenCourt Android aplikacija (.apk)
   - Minimalna verzija: Android 13.0 (API 33)
   - Lokalni keš: SharedPreferences, Firestore offline cache

2. **Firebase Cloud Platform (Backend)**
   - **Firebase Authentication** — upravljanje korisnicima i autentifikacija
   - **Cloud Firestore** — NoSQL baza podataka u realnom vremenu
   - **Firebase Hosting** — opciono za buduće web verzije
   - **Google Cloud Functions** — opciono za serverless logiku (buduće verzije)

3. **Eksterni servisi**
   -**Google Maps API** — prikaz mapa i lokacija klubova
   - **Google Places API** — autocomplete za unos adresa
   - **Geocoding API** — pretvaranje adresa u GPS koordinate

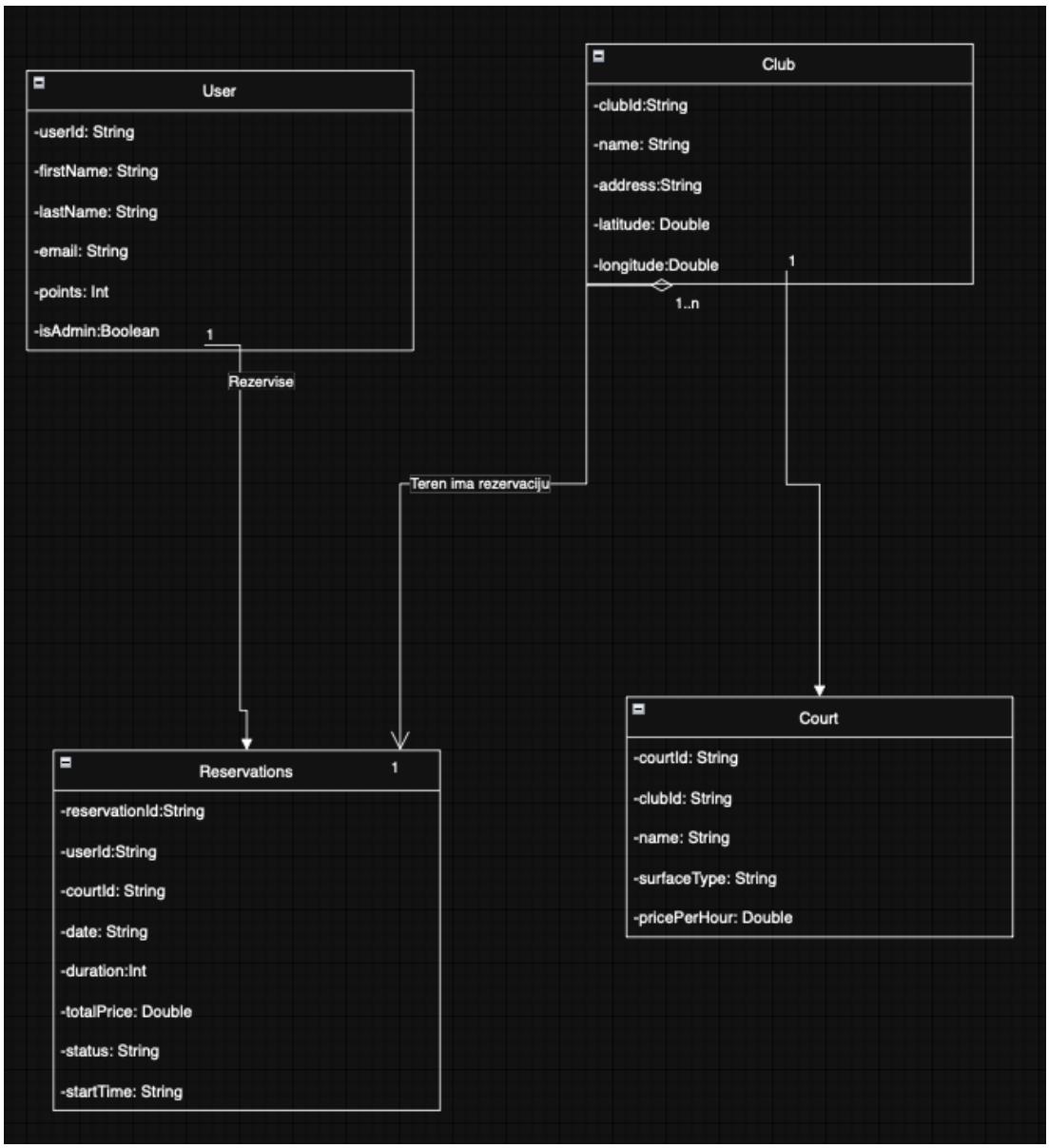
**Komunikacioni protokoli:**

- **HTTPS** — sva komunikacija između aplikacije i Firebase servera
- **WebSocket** — real-time sinhronizacija Firestore podataka

- **Deployment model:**

- Aplikacija se distribuira kao APK fajl ili putem Google Play Store-a
- Firebase projekat se kreira u Firebase Console-u
- Konfiguracioni fajl `google-services.json` se dodaje u Android projekat
- Firestore Security Rules definišu pristupna pravila za podatke

## 10. Pogled na implementaciju sistema




## 11. Performanse

- **Vreme pokretanja aplikacije:** ≤ 2 sekunde
- **Vreme učitavanja liste klubova:** ≤ 3 sekunde
- **Vreme kreiranje rezervacije:** ≤ 1 sekunda
- **Real-time sinhronizacija poruka:** ≤ 1 sekunda
- **Veličina APK fajla:** ~15-20 MB
- **Potrošnja memorije:** ~50-100 MB RAM-a

**Strategije optimizacije:**

- Lazy loading za velike liste (pagination)
- Image caching sa Glide bibliotekom
- Firestore offline persistence za brži pristup podacima
- Debouncing za search funkcionalnost
- RecyclerView sa ViewHolder pattern-om

## 12. Kvalitet

**Metrički kvaliteta:**

- **Pouzdanost:** 99.95% uptime (Firebase SLA)
- **Dostupnost:** 24/7 pristup aplikaciji
- **Skalabilnost:** Do 10,000 korisnika bez degradacije
- **Sigurnost:** HTTPS komunikacija, Firebase Security Rules, šifrovane lozinke
- **Upotrebljivost:** Intuitivni UI, Material Design guidelines, srpski jezik

**Mere kvaliteta:**

- Unit testovi za business logiku
- Integration testovi za Firebase interakciju
- Manual QA testiranje na različitim uređajima
- Firebase Crashlytics za praćenje grešaka
- Firebase Analytics za praćenje korisničkog ponašanja